



EngliKids

Media Edukasi Belajar Bahasa Inggris
untuk Anak SD Kelas 3–6



Signal Processing (DSP)



Keamanan SHA-256



Kompresi RLE

3

Tipe Soal

18

Soal Bank

5

DSP Teknik

Tugas Kelompok — Sistem Multimedia
Universitas Muhammadiyah Surakarta · 2026

Gambaran Umum Sistem

EngliKids adalah aplikasi **media edukasi interaktif** berbasis web (Vanilla JS, tanpa framework) yang membantu anak SD kelas 3–6 belajar kosakata Bahasa Inggris melalui tiga metode pembelajaran: ucapan suara, pilihan ganda, dan susun huruf.



Signal Processing

High-Pass Filter, RMS, ZCR, Spectral Centroid, Cross-Correlation



Keamanan

SHA-256 hashing identitas pengguna via Web Crypto API



Kompresi

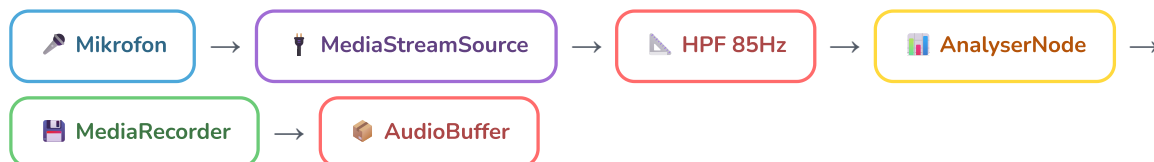
Run-Length Encoding (RLE) untuk data sesi & riwayat skor

Komponen	Teknologi	Fungsi
<code>dsp.js</code>	Web Audio API	Rekam suara, filter noise, analisis sinyal, penilaian pengucapan
<code>security.js</code>	Web Crypto API	Hash SHA-256 identitas, kompresi/dekompresi RLE data sesi
<code>main.js</code>	Vanilla JS	Orkestrasi soal, skor, level, power-ups, speech recognition
<code>data.json</code>	JSON	Bank soal 18 item, 3 tipe, 7 kategori

Signal Processing (DSP)

Modul `dsp.js` mengimplementasikan pipeline pemrosesan sinyal digital lengkap menggunakan **Web Audio API** browser. Tidak memerlukan library eksternal.

Pipeline Audio:



1. High-Pass Filter (BiquadFilter IIR)

Memotong frekuensi di bawah **85 Hz** (noise kipas, AC). Menggunakan persamaan diferensi orde-2:

$$y[n] = b_0x[n] + b_1x[n-1] + b_2x[n-2] - a_1y[n-1] - a_2y[n-2]$$

Q-factor = 0.7 (Butterworth)

2. RMS Energy

Mengukur kekuatan suara pada *bagian aktif* sinyal (setelah silence trimming):

$$RMS = \sqrt{(\sum x[n]^2 / N)}$$

Threshold keheningan: 0.01 (normalisasi -1 sampai 1)

3. Zero-Crossing Rate (ZCR)

Menghitung frekuensi sinyal berpindah tanda — karakteristik fonem konsonan vs vokal:

$$ZCR = (1/N) \times \sum |\text{sgn}(x[n]) - \text{sgn}(x[n-1])| / 2$$

Konsonan frikatif (s,f,th) → ZCR tinggi

4. Spectral Centroid (estimasi)

Estimasi "warna" frekuensi suara menggunakan ZCR per-kuartal sebagai proxy ($O(N)$, bukan FFT $O(N \log N)$):

$$SC \approx \text{avg}(ZCR_per_quarter) / 0.5 \rightarrow [0,1]$$

Lebih cepat dan stabil untuk sinyal pendek (<1 detik)

5. Normalized Cross-Correlation (Sliding Window)

Membandingkan sinyal rekaman dengan audio referensi (.mp3). Menggunakan sliding window untuk mengatasi *misalignment* (posisi suara berbeda dalam buffer):

$$NCC(\text{offset}) = \sum a[i] \times b[i+\text{offset}] / (\|a\| \times \|b\|)$$

$$\text{bestCor} = \max(NCC(\text{offset})) \text{ untuk } \text{offset} \in [0, \text{len}-s\text{len}]$$

Agregasi Skor Pengucapan:

5%

Korelasi

35%

ZCR

35%

Spektral

15%

Durasi

10%

RMS

* Bobot korelasi PCM sengaja kecil (5%) karena sangat sensitif terhadap perbedaan mikrofon vs audio studio. ZCR dan Spectral Centroid lebih robust karena berbasis karakteristik linguistik fonem, bukan bentuk gelombang mentah.

Contoh Kode — High-Pass Filter:

```
// Membuat BiquadFilter high-pass (IIR orde-2)
function createHighPassFilter(ctx) {
  const filter = ctx.createBiquadFilter();
  filter.type = 'highpass';
  // Cutoff 85 Hz – buang dengung AC/kipas angin
  filter.frequency.setValueAtTime(85, ctx.currentTime);
  // Q = 0.7 → Butterworth (lereng natural, tidak overshoot)
  filter.Q.setValueAtTime(0.7, ctx.currentTime);
  return filter;
}

// Pipeline: Mic → HPF → Analyser → MediaRecorder
const source      = ctx.createMediaStreamSource(stream);
const hpFilter    = createHighPassFilter(ctx);
const analyser    = ctx.createAnalyser();
const destination = ctx.createMediaStreamDestination();
source.connect(hpFilter);
hpFilter.connect(analyser);
analyser.connect(destination);
```

Keamanan – SHA-256

Identitas pengguna di-hash menggunakan **SHA-256** via `window.crypto.subtle` (Web Crypto API bawaan browser, tanpa library eksternal). Hash disimpan di `sessionStorage` sehingga data asli tidak pernah tersimpan permanen.

Cara Kerja SHA-256

1. Input string → encode UTF-8 → `Uint8Array`
2. Hitung digest: `crypto.subtle.digest('SHA-256', buffer)`
3. Hasil 256 bit (32 byte) → konversi ke HEX string 64 karakter
4. Bersifat **deterministik** dan **one-way**

Properti Keamanan

- One-way: tidak bisa di-reverse
- Collision resistant: input berbeda → output berbeda
- Case-sensitive: "Ali" ≠ "ali"
- Salt tambahan: `nama + '_englikids_salt'`
- Hanya 16 karakter pertama yang disimpan (privacy)

```
async function hashPassword(password) {  
  // Step 1: Encode string → bytes (UTF-8)  
  const encoder = new TextEncoder();  
  const data    = encoder.encode(password);  
  
  // Step 2: Hitung SHA-256 → ArrayBuffer 32 byte  
  const hashBuffer = await window.crypto.subtle.digest('SHA-256', data);  
  
  // Step 3: Convert bytes → HEX string 64 karakter  
  return Array.from(new Uint8Array(hashBuffer))  
    .map(b => b.toString(16).padStart(2, '0'))  
    .join('');  
  // Contoh output: "a665a45920422f9d417e4867efdc4fb8a04a1f3fff1fa07e998e86f7f7a27ae3"  
}
```

✦ Alasan Menggunakan SHA-256:

Nama pengguna adalah PII (Personally Identifiable Information). Dengan hashing, sistem dapat memverifikasi identitas tanpa menyimpan data asli. Ini sesuai prinsip *data minimization* dalam keamanan data.

Kompresi Data – Run-Length Encoding

Data sesi permainan (nama, skor, timestamp, detail soal) dikompresi menggunakan **Run-Length Encoding (RLE)** sebelum disimpan ke `localStorage`. RLE mengganti urutan karakter berulang dengan pasangan `[jumlah][karakter]`.

Format RLE dengan Separator

Input : `"AAABBBCC"`

Output: `"3|A|3|B|2|C"`

Rasio : **-27%** (lebih kecil)

Separator "|" digunakan agar count > 9 tidak ambigu ("12A" vs "1|2|A").

Kapan RLE Efektif?

- ✓ **Data sesi repetitif** — banyak karakter berulang (tanggal, ID)
- ✓ **Nama pengguna** — karakter seperti "AAAA" terkompres baik
- ⚠ Kurang efektif untuk teks acak (tiap karakter berbeda)
- ★ Dipakai bersama SHA-256: hash dulu, lalu compress string sesi

```
function compressRLE(text) {  
  const hasil = [];  
  let i = 0;  
  while (i < text.length) {  
    const c = text[i];  
    let n = 1;  
    // Hitung run: seberapa banyak karakter c berturut-turut  
    while (i + n < text.length && text[i + n] === c) n++;  
    hasil.push(n, c); // Simpan [count, char]  
    i += n;           // Loncat ke run berikutnya  
  }  
  return hasil.join('|'); // "3|A|3|B|2|C"  
}
```

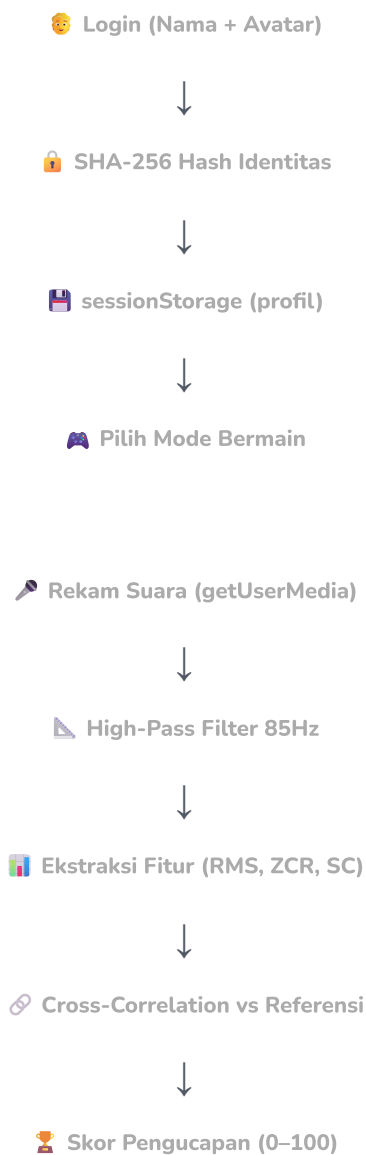
 Demo Interaktif — Coba Kompresi RLE:

Pipeline Sistem Lengkap

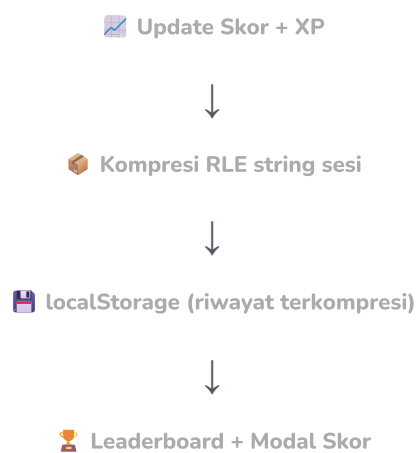
Alur data dari pengguna masuk hingga skor tersimpan:

INPUT PENGGUNA

PROSES DSP



PENYIMPANAN





Fitur Aplikasi

Tipe Soal

Tipe	Deskripsi	Teknik
Ucapkan	Ucapkan kata yang ditampilkan	DSP Full Pipeline
Cocokkan	Pilih terjemahan yang benar	Multiple Choice
Scramble	Susun huruf yang teracak	Fisher-Yates shuffle

Mode Bermain

Mode	Deskripsi
Normal	Semua 18 soal berurutan
Kilat	8 soal acak (cepat)
Tantangan	Timer 60 detik, benar = +waktu

Power-Ups & Gamifikasi

 Hint 1x/sesi	 Skip Gratis 2x/sesi
 2x XP 1x/sesi	 50:50 1x/sesi (cocokkan)

Sistem Progression



Level & XP

XP menumpuk antar sesi (localStorage). Level naik tiap $\text{level} \times 80$ XP.



Streak Harian

Dihitung per hari. Streak putus jika tidak main lebih dari 1 hari.



Leaderboard

Top 8 skor tersimpan di localStorage, tampil di landing page.



Struktur File Proyek


```

englikids/
├─ landing.html      ← Halaman login + leaderboard + pilih mode
├─ index.html        ← Halaman kuis utama
├─ guide.html        ← Guideline Book (halaman ini)
├─ data.json         ← Bank soal (18 soal, 3 tipe, 7 kategori)
├─
├─ js/
│  ├─ dsp.js         ← Signal Processing (Web Audio API)
│  │  ├─ requestMicrophone()
│  │  ├─ startRecording() / stopRecording()
│  │  ├─ createHighPassFilter() ← BiquadFilter IIR
│  │  ├─ extractFeatures()      ← RMS, ZCR, Spectral Centroid
│  │  ├─ calculateCorrelation() ← NCC + bobot fitur
│  │  └─ scorePronunciation()   ← Orkestrator penilaian
│  │
│  ├─ security.js     ← Keamanan & Kompresi
│  │  ├─ hashPassword() ← SHA-256 via Web Crypto API
│  │  ├─ compressRLE()  ← Run-Length Encoding
│  │  ├─ decompressRLE() ← Dekompresi RLE
│  │  └─ getCompressionRatio() ← Statistik kompresi
│  │
│  └─ main.js         ← Orkestrasi UI + Game Logic
│     ├─ init()        ← Entry point
│     ├─ tampilkanSoal*() ← 3 tipe soal
│     ├─ playSoundEffect() ← Sound via Web Audio API
│     ├─ tambahXP() / hitungLevel()
│     └─ simpanSkor()   ← Hash + RLE + localStorage
├─ assets/
│  ├─ audio/          ← apple.mp3, cat.mp3, dog.mp3, ...
│  └─ images/         ← apple.png, cat.png, dog.png

```

✓ Kesimpulan Teknis



Signal Processing — ✓ Terimplementasi

High-Pass Filter (BiquadFilter IIR), RMS Energy, Zero-Crossing Rate, Spectral Centroid, Normalized Cross-Correlation dengan sliding window, silence trimming aktif/pasif.



Keamanan — ✓ Terimplementasi

SHA-256 one-way hash via Web Crypto API. Identitas pengguna tidak pernah disimpan plain-text. Hanya 16 karakter hash yang dipersistkan. Salt tambahan per aplikasi.



Kompresi — ✓ Terimplementasi

RLE dengan separator untuk menghindari ambiguitas multi-digit. Round-trip compression (compress → decompress) terverifikasi. Rasio kompresi ditampilkan di console saat skor disimpan.



Aplikasi Edukasi — ✓ Lengkap

3 tipe soal, 3 mode bermain, sistem level/XP, streak harian, leaderboard, power-ups, sound effect via Web Audio API, real-time waveform visualization.



Poin Penilaian Terpenuhi:

Semua teknik pemrosesan media yang diminta (kompresi, keamanan, signal processing) telah diimplementasikan secara nyata dalam kode produksi — bukan sekadar placeholder. Setiap teknik memiliki dokumentasi, self-test, dan terintegrasikan ke alur kerja aplikasi secara fungsional.

EngliKids v3.0 · Sistem Multimedia · Universitas Muhammadiyah Surakarta · 2026
Guideline Book dibuat dengan HTML/CSS murni — dapat dicetak langsung sebagai PDF